

XAI Mini Project

Anurag Pacholi and Kishan Nadageri

University of Paderborn, 33100 Paderborn, Germany
{apacholi, kishan}@mail.uni-paderborn.de

Abstract. Although Graph Neural Networks (GNNs) are highly effective tools for data analysis, their inner workings can be difficult to interpret, often earning them the "black box" label. This lack of transparency can make it hard to trust their decisions. This project addresses this challenge by exploring how to explain the predictions made by two popular GNN models: GraphSAGE[2] and the simpler Simple Graph Convolution (SGC)[5]. We trained these models to classify relationships within a custom-built family dataset[1].

To understand the reasoning behind the models' predictions, we used an advanced explanation tool called EvoLearner[3]. The main purpose of EvoLearner is to analyse the GNNs' outputs and generate clear, human-readable rules that explain their logic. It effectively translates the complex calculations of the GNNs into simple, understandable concepts based on the family structures in our data.

Our findings demonstrate that this combined approach is highly effective. By pairing powerful GNNs like GraphSAGE and SGC with an explanatory tool like EvoLearner, we can successfully clarify why a model reaches a particular conclusion. This work shows a practical method for making complex AI systems more transparent, which is essential for building trust and gaining deeper insights into the patterns that drive their predictions.

Keywords: GraphSAGE · EvoLearner · Simplified Graph Convolution.

1 Introduction

1.1 The need of AI Explanations

In recent years, Artificial Intelligence (AI) has become a major part of our daily lives. We see it in the movie recommendations we get on Netflix, the navigation apps that help us find the quickest route, and even in advanced medical systems that help doctors identify diseases. As these AI systems become more powerful and are used to make increasingly important decisions, a significant challenge has emerged: we often don't understand how they arrive at their conclusions.

Many advanced AI models operate like a "black box." We can input data and receive an output or a prediction, but the complex processes happening inside remain hidden. This lack of transparency is a serious issue. If we don't know why an AI model made a particular decision, how can we trust it? How can we

be sure it is fair and unbiased? And if it makes a mistake, how can we fix the problem if we don't understand the source of the error? This challenge has led to the development of a new and important field in computer science known as Explainable AI (XAI). The main goal of XAI is to create methods and tools that can make the decisions of AI systems clear and understandable to humans.

1.2 GNN: Graph Neural Networks

One of the most powerful types of AI today is the Graph Neural Network (GNN). GNNs are specially designed to work with data that is structured as a network of connections, or a "graph." Think of a social media network, where people are connected as friends, or a map of a city, where locations are connected by roads. GNNs are excellent at finding hidden patterns and relationships within this kind of connected data.

However, just like other advanced AI, GNNs can also be very complex and difficult to interpret. This project focuses specifically on this problem: how can we explain the predictions made by GNNs? To explore this, we decided to use a type of data that everyone can easily understand: a family tree. A family tree is a perfect example of graph data, with individuals connected by relationships like "parent," "child," or "sibling."

1.3 Our Aim and Objective

The main goal of this mini-project is to demonstrate how XAI techniques can be used to explain the predictions of GNN models in a simple and effective way. We chose a straightforward task: training a GNN to identify who in a family tree holds the role of an "uncle." We selected this task because the logical rule for being an uncle is well-known to humans (a person is an uncle if they are the brother of one of someone's parents). This allows us to easily check whether the explanations generated by our XAI tool are correct and make logical sense.

To achieve our goal, we used two different GNN models: GraphSAGE and Simple Graph Convolution (SGC). After training these models, we used an explanation tool called EvoLearner to analyse their predictions. EvoLearner is designed to generate simple, rule-based explanations that describe the logic the model has learned.

Our key objectives were:

1. To train both GraphSAGE and SGC models to accurately classify the "uncle" relationship in a family dataset.
2. To apply the EvoLearner to generate explanations for the models' predictions.
3. To evaluate these explanations.

2 Data Analysis

2.1 Dataset Overview: The Family Dataset

The dataset used in this project is derived from the OntoLearn research paper and consists of an OWL (Web Ontology Language) file representing family relationships. The ontology defines various family roles (e.g., Father, Mother, Son, Daughter) and their hierarchical relationships, along with object properties such as `hasChild`, `hasParent`, and `married`. The dataset includes multiple family structures with individuals explicitly linked through these properties.

2.2 Key Characteristics

– Classes and Hierarchy:

1. The ontology defines a structured hierarchy of family roles, with **Person** as the root class.
2. Subclasses include **Male**, **Female**, **Parent**, **Child**, **Grandparent**, and more specialized roles like **Brother**, **Sister**, **Granddaughter**, etc.
3. Disjoint classes ensure that individuals cannot belong to both **Male** and **Female** simultaneously.

– Object Properties:

1. **hasChild**: Links parents to their children.
2. **hasParent**: Links children to their parents.
3. **hasSibling**: Defines sibling relationships.
4. **married**: Indicates marital relationships between individuals.

– Data Splits:

1. Training Set: 60% of the complete dataset is used as the training data.
2. Test Set: 40% of the complete dataset is used as the test data.
3. There are a total of 38 positive examples and a total of 38 negative examples used as the ground truth (available in OntoLearn's Github).

– Observations about the dataset:

1. Example families include F1, F2, ..., F7, ..., each with distinct structures (e.g., single-parent, multi-generational).
2. **Complex Relationships**: The dataset includes multi-generational families with intricate relationships (e.g., grandparents, grandchildren).
3. **Consistency**: The ontology enforces logical constraints (e.g., a Father must be Male).

4. **Scalability:** The dataset is sufficiently large to test the EvoLearner’s ability to infer complex family structures.

– **Suitability for EvoLearner:**

1. The dataset’s hierarchical and relational nature aligns well with EvoLearner’s ability to learn class expressions.
2. The explicit property assertions provide clear ground truth for evaluating learned rules.

3 Model Training and Evaluation

3.1 SGC (Simplified Graph Convolution)

The Simplified Graph Convolution (SGC) model is a streamlined variant of Graph Convolutional Networks (GCNs) that reduces computational complexity by removing nonlinear activations between layers. SGC works by performing a series of graph smoothing operations followed by a linear transformation. This makes it computationally efficient while still capturing graph structure effectively.

– **Key Features:**

1. **Efficiency:** Removes intermediate nonlinearities, reducing training time.
2. **Scalability:** Performs well on large graphs due to its simplified architecture.
3. **Applicability:** Suitable for tasks where graph structure is crucial but deep feature learning is not required.

3.2 GraphSAGE (Graph Sample and Aggregated Embeddings)

GraphSAGE is an inductive graph neural network framework that learns node embeddings by sampling and aggregating features from a node’s local neighborhood. Unlike transductive models, GraphSAGE can generalize to unseen nodes, making it useful for dynamic graphs.

– **Key Features:**

1. **Inductive Learning:** Can generalize to new, unseen nodes.
2. **Neighborhood Aggregation:** Uses aggregator functions (e.g., mean, LSTM, pooling) to combine neighbor features.
3. **Flexibility:** Works well with both homogeneous and heterogeneous graphs.

4 Evaluation Metrics

To assess the performance of both models, we use four standard classification metrics:

1. Precision

Measures the proportion of correctly predicted positive instances out of all predicted positives.

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

- **Interpretation:** High precision means fewer false positives (incorrectly predicted relationships).

2. Recall (Sensitivity)

Measures the proportion of actual positives correctly identified by the model.

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

- **Interpretation:** High recall means the model captures most true relationships.

3. F1-Score

The harmonic mean of precision and recall, providing a balanced measure.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Interpretation:** Useful when class imbalance exists; balances precision and recall.

4. Accuracy

Measures the overall correctness of predictions.

$$\text{Accuracy} = \frac{\text{True Positives (TP)} + \text{True Negatives (TN)}}{\text{Total Predictions}}$$

- **Interpretation:** Best for balanced datasets; may be misleading if classes are imbalanced.

5 Explanation Model: EvoLearner

EvoLearner is an evolutionary approach designed to learn description logic (DL) concepts from positive and negative examples in knowledge graphs. Unlike black-box models, which often lack interpretability, EvoLearner generates human-readable and explainable concepts in the form of description logic expressions. These concepts can be directly mapped to OWL class expressions, making them compatible with the Semantic Web ecosystem. The model’s primary goal is to classify nodes in knowledge graphs, such as predicting entity types or identifying promising drug candidates, while providing globally explainable rules that can be validated and enriched with domain knowledge.

Key Components of EvoLearner:

5.1 Evolutionary Algorithm Framework

EvoLearner employs an evolutionary algorithm to iteratively refine and optimize description logic concepts. The algorithm operates on a population of candidate concepts, represented as abstract syntax trees, where each node corresponds to a logical operator or terminal (e.g., atomic concepts, data properties). The evolutionary process involves:

- **Initialization:** Generating an initial population of concepts using biased random walks (discussed below).
- **Selection:** Choosing the fittest individuals based on accuracy and concept length.
- **Crossover:** Combining parts of two parent concepts to produce offspring.
- **Mutation:** Randomly modifying concepts to introduce diversity.

5.2 Biased Random Walk Initialization

A novel contribution of EvoLearner is its initialization method, which starts from positive examples in the knowledge graph and performs biased random walks to construct initial concepts. This approach ensures that the generated concepts are grounded in the actual data, avoiding the combinatorial explosion typical of top-down methods. The steps include:

- Selecting a positive example and one of its types, weighted by frequency.
- Traversing outgoing relations (object or data properties) from the example, up to a predefined limit (maxT).
- Translating the traversed paths into description logic concepts, such as existential restrictions or value constraints for data properties.
- Completing incomplete subconcepts by randomly selecting types or extending paths further.

This method significantly improves the quality and diversity of the initial population, leading to faster convergence and better performance compared to traditional initialization techniques like Grow or Full.

5.3 Support for Data Properties

EvoLearner enhances the handling of data properties (e.g., numeric or Boolean values) by maximizing information gain when determining splits for numeric thresholds. This is inspired by decision tree induction and involves:

- Precomputing splits for each data property based on the distribution of positive and negative examples.
- Using these splits during initialization and mutation to generate meaningful value restrictions (e.g., $d \geq v$ or $d \leq v$).
- Ensuring that the splits optimally separate positive and negative examples, improving predictive performance.

5.4 Fitness Function and Bloat Control

The fitness of each concept is evaluated using a combination of accuracy and concept length:

- **Accuracy:** Measures the proportion of correct predictions on the training set.
- **Bloat Control:** Penalizes overly long concepts to prevent overfitting and improve generalization. The fitness function is defined as:

$$\text{fitness}(C) = \text{accuracy}(C) - \alpha \cdot \text{length}(C),$$

where α is a weighting factor. Additionally, a static depth limit is enforced to further control complexity.

Advantages of EvoLearner

Explainability

EvoLearner generates concepts in description logic, which are inherently interpretable. These concepts can be verbalized into natural language, enabling domain experts to understand, validate, and refine the rules. For example, the concept for “Uncle” in the Family dataset is expressed as:

$$\text{Male} \sqcap (\exists \text{married}.\exists \text{hasSibling}.\text{Parent}) \sqcup (\exists \text{hasSibling}.\text{Parent}),$$

which clearly defines the conditions for an uncle.

Performance

EvoLearner outperforms state-of-the-art baselines like CELOE, OCEL, Aleph, and SPaCEL on most benchmarking datasets. It achieves higher accuracy and F_1 -scores while generating shorter and more generalizable concepts. The ablation study confirms that both the random walk initialization and data property support are critical to its success.

Scalability

By focusing on paths that actually appear in the data, EvoLearner avoids the inefficiencies of top-down approaches that explore irrelevant parts of the search space. This makes it particularly effective for large knowledge graphs with many object properties.

Flexibility

The model supports a wide range of description logic constructs, including negation, union, intersection, existential/universal restrictions, and cardinality constraints. This flexibility allows it to handle diverse real-world applications, from molecular property prediction to entity classification.

6 Conclusion

We implemented EvoLearner, an evolutionary approach for learning description logic concepts from knowledge graphs. Our method combines:

- **Model Implementation:** Implementation of SGC and GraphSage model.
- **Explainer Implementation:** Implementation of EvoLearner on the predictions of the model.
- **Evaluations:** Evaluating the predictions of the models as well as the explainers.

To ensure benchmarking, we adopted metrics from **EDGE: Evaluation Framework for Logical vs. Subgraph Explanations for Node Classifiers on Knowledge Graphs** [4], which presents evaluations across various datasets, explainers and models.

6.1 Results on the Family Dataset with SGC Model:

- **Prediction Accuracy:** 1.00
- **Prediction F_1 -score:** 1.00
- **Prediction Precision:** 1.00
- **Prediction Recall:** 1.00

- **Explanation Accuracy:** 0.8438
- **Explanation F_1 -score:** 0.9231
- **Explanation Precision:** 0.7500
- **Explanation Recall:** 0.8276

- **Explanation:** Best Achieved Concept:
 Uncle = "Male $\sqcap$ ($\exists$ hasSibling.Parent $\sqcup$ $\exists$ married. $\exists$ hasSibling.Parent)"

6.2 Results on the Family Dataset with GraphSAGE Model:

- **Prediction Accuracy:** 1.00
- **Prediction F_1 -score:** 1.00
- **Prediction Precision:** 1.00
- **Prediction Recall:** 1.00

- **Explanation Accuracy:** 0.9375
- **Explanation F_1 -score:** 1.00
- **Explanation Precision:** 0.8750
- **Explanation Recall:** 0.9333

- **Explanation:** Best Achieved Concept:
 Uncle = "Male $\sqcap$ ($\exists$ hasSibling.Parent $\sqcup$ $\exists$ married. $\exists$ hasSibling.Parent)"

Contributions of Team Members

The team contributions were distributed as follows:

- **Anurag Pacholi:**
 - Prepared training and testing datasets.
 - Implemented the EvoLearner Explainer.
 - Conducted evaluations and calculated performance metrics.
 - Authored the report.

- **Kishan Nadageri:**
 - Implemented the GNN models.
 - Conducted evaluations and calculated performance metrics.
 - Maintained an organized, modular codebase.

Both of us contributed to the report revisions and approved the final version of both the report and the code.

References

1. Demir, C., Baci, A., Kouagou, N.J., Sieger, L.N., Heindorf, S., Bin, S., Blübaum, L., Biggerl, A., Ngomo, A.C.N.: Ontolearn—a framework for large-scale owl class expression learning in python. *Journal of Machine Learning Research* **26**(63), 1–6 (2025), <http://jmlr.org/papers/v26/24-1113.html>
2. Hamilton, W., Ying, Z., Leskovec, J.: Inductive representation learning on large graphs. *Advances in neural information processing systems* **30** (2017)
3. Heindorf, S., Blübaum, L., Düsterhus, N., Werner, T., Golani, V.N., Demir, C., Ngonga Ngomo, A.C.: Evolearner: Learning description logics with evolutionary algorithms. In: *Proceedings of the ACM Web Conference 2022* (2022). <https://doi.org/10.1145/3485447.3511925>

4. Sapkota, R., Köhler, D., Heindorf, S.: Edge: Evaluation framework for logical vs. subgraph explanations for node classifiers on knowledge graphs. In: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management. pp. 4026–4030 (2024)
5. Wu, F., Souza, A., Zhang, T., Fifty, C., Yu, T., Weinberger, K.: Simplifying graph convolutional networks. In: Chaudhuri, K., Salakhutdinov, R. (eds.) Proceedings of the 36th International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 97, pp. 6861–6871. PMLR (09–15 Jun 2019), <https://proceedings.mlr.press/v97/wu19e.html>